Thor-Soar Agent Documentation

The Thor-Soar agent serves as our baseline for developing agents in the AI2Thor environment.

- **Environment:** We have extended the environment to include “machines” that operate over time to change the properties of objects in the world (mostly for cooking): microwave, stove, toaster (toasts), coffeemaker (makes coffee), dishwasher (cleans dishes).
- **Perception:** We are experimenting with three levels of perception:
  - perfect, omniscient perception drawn directly from the simulator;
  - a frustum culled perception, with occlusion (can’t see into containers that are closed, but still derived from the simulator’s ground truth;
  - a pixel-based recognition of objects using YOLO (developed by J. Boggs).
- **Action:** We use the primitive action commands provided by AI2Thor. There is an underlying navigation system that moves the agent close to objects. It does cause some grief, and needs to be upgraded at some point so that an agent moves and turns to get a front on facing to the objects it approaches (such as a cabinet).
- **Innate Task-independent Operators:** These support MEA operator subgoaling, grounding, etc. Some use MEA, but most are proposed when they need to do their thing. No plans to learn via ITL. No further discussion below. No changes.
- **Innate Task-Action Operators (TAO):** These are the primitive domain-specific operators, and each has a single external action that can be performed in the world. No substate implementation. No plans to learn via ITL.
  - These operators all have an MEA proposal rule and an internal execution rule. Some have multiple proposals and additional selection rules.
  - All are "instantaneous" from the viewpoint of the agent, except for turn-on, which when applied to some objects (coffeemaker, stoveknob, microwave, toaster) initiates dynamics in the environment. Turn-on creates a future-event structure that predicts when something is cooked, etc.
- **Simple Composite Task Operators (sCTO):** These are domain specific. These should be learnable via ITL. They are implemented as operator proposal rules, which have precondition desireds that invoke, via MEA, other composite and task-action operators to achieve the preconditions, which essentially implement them. They do not directly initiate external actions, and do not have internal implementation rules for look-ahead. Discard and store are examples in the current agent.
- **Complex Composite Task Operators (cCTO):** These are domain specific. These should be learnable via ITL. They are "complex" because they require operators (TAO, sCTO,

cCTO) in a substate to implement them. That substate arises *after* all the preconditions are satisfied.

- Current examples are cook-microwave, toast, make-coffee, cook-stove. All of those have only turn-on as a substate implementation operator.
- They have the same structure as sCTOs but also have an ^effect-desireds structure that specifies what should be achieved once the preconditions are satisfied. When all the preconditions are satisfied, the ^effect-desireds structure is copied to the resulting operator implementation substate as ^desireds. Effects can include sequencing (through ^next-desireds), disjunction, and temporal extent, which are not expressible in the preconditions.
  - This structure can include a set of ^desired augmentations, and for all of the current CTOs, it it is that their "machine" is ^activated true.
  - The ^next-desireds structure provides a way to directly represent step-by-step procedures. .
  - All the infrastructure for interpreting these reuses what is already there for processing desireds/desired.
- They do not require additional internal application rules for look-ahead, because they eventually bottom out in Task-Action Operators.
- The knowledge that creates ^future-events is tied to them only indirectly - nothing about future-events needs to be learned through ITL except that an object must be activated via turn-on. All the knowledge about creating future-events is part of the implementation of turn-on.
- We might want to look at having the ^effect-desireds created by a separate rule(s).

- **Other Innate Task Knowledge:** There are other places where some task knowledge is sprinkled in, and a goal for me (not the agent) is to get that cleaned up. Examples of this type of knowledge include elaborations for determining which specific properties or relations are achieved (or unachieved).
- **Reasoning and Planning:** The agent uses:
  - means-ends analysis and operator subgoaling (MEA-OS) to limit search;
  - multiple forms of search control knowledge to further limit search. If such knowledge is missing, the agent falls back on internal search.
  - iterative deepening look-ahead planning to find temporally shortest plans;
  - temporally extended actions (such as turning on the microwave);
  - hierarchical/composite operators;
- **Learning:** chunking over look-ahead, and learns preference rules. More work is needed to expand the use of chunking to include learning operator implementation rules. Currently, it

does not use episodic memory, online semantic learning, or reinforcement learning, all of which will be added when we do long-duration tasks.

- **Language Interaction:** The agent uses both a Soar-based parser (Lucia) and a large language model (LLM) to support natural language commands. [This document does not describe the NL/ITL/LLM capabilities - yet.]
- **Interactive Task Learning:** This is our research focus, but it is not described in this document except when we discuss task and operator representations and how they impact task learning.

This document details most aspects of the agent, organized by the representations and the processing it uses to perform a task. Different concepts are interconnected, so the presentation has redundancy. In each section, I include shortcomings and planned extensions of the current approach. Conventions for the agent's representations are also described in the Soar Conventions document.

There are separate sections on Means-Ends problem solving and Iterative Deepening at the end that try to pull information from across the agent together to explain how these work (and how they interact). There is also a final section on chunking. [These sections have not yet been written 😞]

Top-State Representation

This section describes the major data structures, starting with those in the top state. Listed in order of importance to some degree. Items in bold are important. Others can mostly be ignored. Many of these are shared in substates listed after this section.

The most important structures are:

- the **^scene** and **^world** structures (on the top state and substates): these describe what the agent knows about the task situation. It is the problem-space state.
- the ^**desireds**: these describe what the agent is attempting to achieve - its goals and tasks.
- Other structures on the top state. These are used to control reasoning and also include flags and constants. Not all of these top-state structures are listed below (yet).

Scene [identifier]

The scene includes all relations and objects visible to the agent, as listed on the input-link. This is everything in a room if frustum culling is disabled; otherwise, it is the subset in the view frustum. The attributes listed in attends.attend are copied from the scene to the world structure.

- ^**on/in** [identifier: relation] The attended relations (same identifiers) are copied to the world.
- ^**objects.object** [identifier: object] The attended objects are copied (same identifiers) to the world.

- ^**holding-object** [identifier: object] Either the identifier of the object being held or none.

Example:

```
(I6 ^holding-object H1 ^in I38 ^in I37 ^in I36 ^in I35 ^in I34 ^in I33
^in I30 ^in I24 ^in I20 ^in I19 ^in I16 ^in I15 ^in I14 ^in I13 ^in
I10 ^in I9 ^in I8 ^objects I7 ^on I64 ^on I66 ^on I63 ^on I62 ^on I61
^on I60 ^on I59 ^on I58 ^on I57 ^on I56 ^on I55 ^on I54 ^on I53 ^on
I52 ^on I51 ^on I50 ^on I49 ^on I48 ^on I47 ^on I65 ^on I46)
```

World [identifier]

The world is the set of data structures representing the agent's world state. It does not include state structures that control processing. It is created in the top-state from the ^scene based on those objects that are attended (^attends). The major content consists of relations (^in, ^on ^holding-object) and objects (^objects). A few other attributes are listed later.

A world structure can be:

- a direct reflection of what the agent knows about the current state of the external world as derived from the ^scene (<world> ^real true ^anchored true),
- a copy of the external world that is used for look-ahead search (<world> ^real false ^anchored true),
- a description of objects and relations that need to be grounded to objects and relations in the scene (<world> ^real false ^anchored false).

There are also ^world structures that describe relations and objects to be grounded. These are derived from ^world-context structures in ^desireds. See ^desireds for more on them.

- **^objects** [identifier: objects] A pointer to the set of individual objects (^objects.object). Each object is a representation of an object in the world. The substructure of an object is listed below. In general, for the top-state world, these are only those attended to (see ^attends).
- **^in** and **^on** [identifier: relation] These are relations with the following structure.
  - ^**argument-count** [2] The number of arguments: 2.
  - **^argument1** [identifier: object] The identifier of the contained or on top object.
  - **^argument2** [identifier: object] The identifier of the container object.
  - ^**duplicate-of** [identifier: relation] The identifier of a relation this was copied from if used in a look-ahead search in an evaluation substate. Not used in the real world.
- **^anchored [true/false]** This distinguishes between worlds that are models of the real world - [true] either the real world derived from a scene or copies of those in look-ahead search, or [false] a world that describes constraints on desired objects.
- **^real [true/false]** It is true for the world on the top-state and substates that don't make a copy of it (operator no-changes). If it is false, then it is a copy being used in a look-ahead

state in evaluate-operator substate.

- **^time [integer: milliseconds]** Current time in milliseconds, copied from input-link. When real false, this is computed based on internal models of external operator applications
- ^total-time-minimum-achieve-time [integer: milliseconds]. Estimated total time in milliseconds to achieve all desired. Created in elaborations/time. currently used.

Example when anchored true

```
(W1 ^anchored true ^holding-object H1 ^in I30 ^objects W2 ^on I49 ^on I50 ^on I51 ^on I47 ^on I53 ^on I54 ^on I48 ^on I46 ^on I52 ^real true ^time 450)
```

Example when anchored false

```
(W3 ^anchored false ^in I30 ^objects W2 ^on I49)
```

Desireds [identifier]

An agent's goals are contained in the desireds structure, which has a set of individual desired items. These are conjunctive so that all must be achieved for the goal to be achieved. The desireds structure comes from interactions with the outside world or from an elaboration based on the ^desired-selection flag. The desireds structure is also used in the representation of operators for the operator's preconditions - see operators for details as some processing is a bit different. The desireds structure keeps track of which individual ^desired is achieved or unachieved and how many of each there are. It also provides a way to specify preferences for which desired to pursue over others and allows specification of future desireds to be achieved after this one is (next-desired). Disjunction is also possible.

See elaborations/desireds for many examples.

If all the desired(s) in the top state are achieved, then the agent halts. If in a lookahead search, the state will get **^success** <desireds> added to it.

- **^desired** [identifier: desired] This describes a *single* relation or object property to be achieved. It is important that multiple properties (such as an object being reachable and open) are *not* combined in a single desired condition, as this makes it difficult to have uniform procedures for detecting when all have been achieved. (Need to expand on this set in the future, possibly including things the agent wants to know, tell, ...)
  - **^holding-object** [none/object identifier] The identifier is not the identifier of an object in the world on the state. Its object-handle is a constant that is the same as the object-handle of an object in the state, and any other properties (such as ^class) are used to constrain it, but once again, it is best to have only one such property. Use the ^world-context structure (see below) to further constrain the object.
    - Example: Potato55 must be held

- `(<desireds> ^desired.holding-object.object-handle Potato55 ^desired-count 1)`

- **^in/on** [identifier: relation] This is the same as an ^in relation on the world, except that it has only the object-handle for argument1 and argument2 and not a complete object description. To specify further constraints on those objects (such as one must be cook-status cooked), add an additional object desired with the shared object-handle for each property. If the specific objects are unknown, a fake object-handle will be used that is then constrained by additional information can be provided in the world-context (see below).
  - Example: Apple1 must be in Microwave47
  - `(<desired1> ^in <in1>)`
    `(<in1> ^argument1.object-handle Apple1`
    `^argument2.object-handle Microwave47)`
- **^object** [identifier: object] This partial object description that has a desired property (such as ^reachable true or ^class apple) and usually an object-handle. The desired property is what is being achieved. This should not be a complete object or have multiple properties. The object-handle is used to ground/connect with objects in the world (world.objects.object.object-handle). If the object-handle is missing, some reasoning (grounding) will be required to determine what it is. (knowledge operators, not complete) Additional information to ground the object can be provided in the world-context (see below). The number of features/attributes that can be specified is limited to two - this can easily be expanded (ask John).
  - Example: Potato50 must be sliced
  - `(D10 ^object O80)`

    `(O80 ^object-handle Potato50 ^sliced true)`
- **Disjunction.** To specify disjunctive between two desired{s) (either one can be achieved for the desireds to be achieved), each desired structure in the disjunction has at ^disjunction property of the disjunctive desired.
  - Example: Apple1 must be in Fridge40 or on CounterTop22
  - `(<s> ^desireds <desireds>)`
    `(<desireds> ^desired <desired1> <desired2>`
    `^desired-count 2)`
  - `(<desired1> ^in <in>`
    `^disjunction <desired2>)`
    `(<in> ^argument1.object-handle Apple1`
    `^argument2.object-handle Fridge40)`
    `(<desired2> ^on <on>`

```
            ^disjunction <desired1>)
(<on> ^argument1.object-handle Apple1
      ^argument2.object-handle CounterTop22)
```

- ^**desired-count** [integer] Number of desired structures. Needs to be included when desireds structure is created. Will not be computed as an elaboration.
- ^**achieved** [identifier: desired] The value is the identifier of a desired that has been achieved. Computed in elaborations/achieved-desireds. Note that a desired can flip back and forth from achieved to unachieved. Computing ^achieved and ^unachieved are critical to MEA. A common bug is to see some desired identifier <id> with both ^achieved <id> and ^unachieved <id>.
- ^**unachieved** [identifier: desired] This is the identifier of a desired that has *not* been achieved. Computed in elaborations/achieved-desireds. These are included to avoid negated conditions in chunks.
- ^**unachieved-desireds** [true] Signals that some desireds are not achieved.
- ^**unachieved-desired-count** [integer] The number of desireds that are unachieved. Computed in achieved-desireds. It is used to determine ^unachieved-desireds true.
- ^**next-desireds** [identifier: desireds] This is the set of desireds that should be attempted after these desireds have been achieved. Allows strict ordering of sets of goals.
  - Example: Get Knife44 then put it in Drawer30
  - ```
    (<s> ^desireds <desireds>)
    (<desireds> ^desired <desired1>
                ^desired-count 1
                ^next-desireds <desireds2>)
    (<desired1> ^holding-object.object-handle Knife44)
    (<desireds2> ^desired <desired3>
                 ^desired-count 1)
    (<desired3> ^in <in>)
    (<in> ^argument1.object-handle Knife44
          ^argument2.object-handle Drawer30)
    ```
- **^world-context** [identifier: world] This is a structure that describes objects and relations (in/on/holding) in the current world that are used to ground “fake” object-handles in ^desired structures. Thus, world-context = current world description; desired structures describe the goal state to be achieved. Share object-handles connect them. The world-context specfication is in the same form as the world; however, it is used to constrain the grounding of the objects in a desireds structure - which can include objects in a relation in a desired. Thus, its substructure can include ^objects.object <object> where the <object> has properties of objects in a desired that share the same fake object-handle. It will also have attribute ^anchored false, to indicate it is not grounded/anchored. The example below
    ```

shows that the object to be held should be of class egg (and the object-handle is not yet known).

```
(<s> ^desireds <desireds>)
(<desireds> ^desired.holding-object.object-handle |unanchored-object-egg-handle|
            ^desired-count 1
            ^world-context.objects.object <object-egg>)
(<object-egg> ^class egg
              ^object-handle |unanchored-object-egg-handle|
              ^anchored false)
```

The world-context structure will often be added to a state as a ^world, but it is distinguished by ^anchored false (and no ^real true). This allows rules to match against an ungrounded structure and use it, delaying grounding until it is actually needed.

- ^avoid-desired [identifier: desired] Signals that this desired should not be attempted until all others are. It leads to the creation of a worst preference for operators that attempt to achieve it. The rule for this is in elaborations/search-control.soar as are the rules for the next two below.
- ^preferred-desired [identifier] Signals that this desired should be attempted first. It leads to the creation of a best preference for operators that attempt to achieve it.
- ^preferred-indifferent [identifier: desired] Signals that this desired is indifferent to other operators that attempt to achieve other preferred-indifferent desireds. It leads to the creation of an indifferent preference for any operators that attempt to achieve it.
- ^duplicate-of [identifier: desired] This is the desireds this was duplicated from in a look-ahead search. Need to duplicate it because in the look-ahead, the desired structure is modified (with achieved, etc.).
- ^minimum-achieve-time [integer: milliseconds] Minimum estimated time to achieve the specific desired. Experimental for early cutoff
- ^total-minimum-achieve-time [integer: milliseconds] Total minimum estimated time to achieve all of the desired(s). Is used to cut off search in look-ahead. Experimental..
- object-state-desireds [true] If true, then desireds are created by the operator create-object-state-desired for all objects that are left open or activated according to constants desired-object-states. Only relevant if ^flags.clean-up-on-success true.

Example:

`(D9 ^desired D10 ^desired-count 1 ^minimum-achieve-time M1 ^total-minimum-achieve-time 500 ^unachieved D10 ^unachieved-desireds true ^unachieved-desireds-count 1)`

Objects.object [identifier]

Objects is the attribute on the world, which then has multiple object attributes, one for each object. This is the representation of objects in the world. There are many true/false features that can be associated with each object. These and other features are derived from the

object structures on the io.input.objects.object structure (which has a different structure than world.objects.object. Most of these are created by rules in corresponding folders in the elaborations folder. For example, whether an object is openable is determined in elaborations/open-closed.soar, and whether it is sliceable is in elaborations/sliced.soar.

- Properties that specify whether something is possible with an object. Usually, they only exist if the value is true.
  - activable, breakable, cookable, cooks, dirtyable, fillable, graspable, openable, reachable, sliceable, toggleable.
- For those properties that are possible, the value of that property. The first list is true/false. Those with other values after.
  - activated, broken, contains-liquid, dirty, open, reachable, sliced, toggle
  - cook-state [raw, cooked, burnt]
  - liquid [empty, coffee, water, wine]
- ^**object-handle** [symbol] The unique symbol for this object.
- ^origin-object-handle [symbol] The object-handle of the object this object was derived from. Currently, this is only for object slices.
- ^**class** [enumeration] The type of object, such as vase, apple, floor, ...
- ^**receptable** [enumeration: surface/receptacle] Seems like it overlaps with type.
- ^**type** [location/container] Only these two types. Seems like it overlaps with type.
- ^derived-from-input [identifier: input-object] This is the object on the input-link that the information for this object was derived from. Useful in debugging but not used by agent reasoning.
- ^duplicate-of [identifier: object] The original object this was copied from in a look-ahead search.
- ^material [enumeration] The type of material the object is, such as food, glass, ... I use category more than this.
- ^mass [string: kilograms?] Not used.
- ^ms-remaining [integer: milliseconds] The number of milliseconds an object will remain activated, such as the microwave while it is cooking.
- ^temperature [enumeration: room-temp/?] Not used.

Example

```
(N38 ^activatable false ^affordance grabbable ^breakable false
^category knife ^cookable false ^derived-from-input O41 ^dirtyable
false ^fillable false ^graspable true ^mass |0.18| ^material plastic
^material metal ^object-handle Knife40 ^reachable false ^sliceable
false ^temperature room-temp)
```

Attends.attend [string: object-handle]

Attends is the attribute on the top-state that has an augmentation for each attend. Each attend is the object-handle of an object that should be added to ^world. objects. Attend structures are created in elaborations/attend-objects. Below are (most of) the object-handles that are attended to:

- objects in the scene that are in a desired, including desired.object, desired.on/in.argument1/argument2, desired.holding-object.
- objects that are in the scene that cook if there is a desired.object.cook-state cooked
- objects of category knife if there is a desired.object.sliced true. We should probably add a property slices.
- objects that are receptacle contain objects that have an attend.
- objects that are reachable and a receptacle if there is a desired.holding-object.object-handle none. This is a bit of a hack but a place available if need to put down what is in the hand.
- objects that are sinkbasin or faucet if have a desired be clean (dirty false). Once again a bit of a hack.
- objects that have origin-object-handle of an object that is to be sliced. This makes the sliced results available in the world as object(s).

Example:

```
(A2 ^attend CounterTop19 ^attend Potato50 ^attend Knife40 ^attend Drawer27)
```

Problem-space [identifier]

The problem space name specifies the context for which operators are available in a given state. Often it also includes flags that create a copy of a state in look-ahead evaluate-operator substates. Those flags are ignored in Thor-Soar (they are set so there is no default copying); there is a set of rules that copy the world and desired.

- ^**name** [string: Thor-Soar]
- ^internal-actions [true/false] If true, it will simulate the actions instead of sending commands to AI-Thor via the output link. It is set to true in look-ahead substates. I have not recently tested true for top-state, which is set by a flag (internal-simulation).

Example:

```
(P508 ^default-desired-copy no ^default-operator-copy no ^default-state-copy no ^dont-copy top-state ^internal-actions false ^name thor-world)
```

Constants

These are constants that are used during agent reasoning. They differ from flags in that they don't turn different types of processing on and off, but determine the values used in processing. Probably the timing ones should on the input-link derived from their values in the .config file.

- ^action-time [integer: millisecond] How long an action will take to execute in the world. Not correct for approach.
- ^food-cook-time [integer: millisecond] How long an object takes to cook. Should be specialized to specific object types (as specified in .config)
- ^decision-time [integer: millisecond] How long a decision takes in simulated time. Should be as specified in .config
- ^desired-object-state [identifier] A set of object properties that should become desired after all specified desireds are achieved. Controlled by a flag as to whether this happens after all specified desireds are achieved. Controlled by a flag.
  - Example:
  - `(D2 ^activated false ^open false)`
- ^desidered-ordering [identifier] A set of numbers associated with desired structures that are used to order which desireds are attempted first. Lower numbers are preferred. Controlled by a flag.
  - Example:
  - `(D1 ^desired-ordering D4 D5 D6 D7 D8 D3)`

    `(D4 ^relation in ^value 1)`

    `(D5 ^object-property open ^value 2)`

    `(D6 ^object-property closed ^value 2)`

    `(D7 ^relation holding-object ^value 3)`

    `(D8 ^object-property reachable ^value 4)`

    `(D3 ^relation on ^value 1)`

Examples:

`(C85 ^action-time 500 ^decision-time 50 ^desired-object-states D2 ^desired-orderings D1 ^food-cook-time 15500)`

Flags

These determine the specific processing. In elaborations/flags-initialization.soar. Defaults are in bold.

- ^attend-all [true/**false**] If true, everything in the scene is attended to and is available in world.

- ^clean-up-on-success [true/**false**] If true, will clean up the world by deactivating objects (not the lights) and closing open containers.
- ^desired-preferences [**true**/false] If true, will use the preferences specified to desireds for choosing which operators to select in operator subgoals.
- ^desired-source [elaboration] If elaboration, then the value of desired-selection is used. This is only set if ^user is JEL.
- ^desired-selection [string] This is used with desireds-source is elaboration. There are a set of rules in elaborations/desireds that then creates the appropriate desireds structure.
- ^forget-unattended [true/**false**] If true, structures on the world will be removed if there is no longer an attend for them (or their derivation). True value not really tested.
- ^gps-desired-ordering [**true**/false] If true, constants.desidered-ordering are used to choose between operators proposed via mea.
- ^internal-simulation [true/**false**] If true, will simulate top-state task operators instead of sending commands to output-link. Has not be tested recently.
- look-aheading-planning [**true**/false] If true, using selection space, etc. to determine which operator to select if ties. If false, all operators are made indifferent - useful for random forward search possibly supplemented by reinforcement learning.
- mea-operator-selection [**true**/false] If true, uses operator proposal rules that are based on MEA. If false, operators are proposed when their preconditions are satisfied.
- ^rl [true/**false**] If true, uses reinforcement learning. Not supported. Need to add RL rules (via templates or gp). Hard to do with the state representation we have.
- ^use-timers [**true**/false] If true, reasons about time in look-ahead.
- ^user [string] This should be some string identifier of the user that is then used to specialize the flags and other things. Right now, desired-source is the flag that is determined by ^user in the agent code base (^user JEL leads to ^desireds-source elaborations). It should be set in each user's file that sources the agent.

Example:

```
(F1 ^attend-all false ^clean-up-on-success false ^desired-preferences true ^desired-selection slice-potato ^desireds-source elaboration ^forget-unattended false ^gps-desired-ordering true ^internal-simulation false ^look-ahead-planning true ^mea-operator-selection true ^rl false ^use-timers true ^user JEL)
```

Internal-actions [true/false]

This determines whether task operators are internally simulated or are executed via the output-link.

Innate Task-Action Operators

This section describes the format of innate task-action operators. These operators create a structure on the output-link to initiate an action in the environment. They do not have any operator implementation substates. It is not expected that these are learned through instruction; rather, they are innate. They are the building blocks for creating composite task operators, which do not directly issue action commands but are composed of task-action operators. There are other innate operators that perform internal functions (such as grounding, desireds management, scene management, attention, ...). [There are some operators for exploration and scanning that might be hybrid for now. I need to get around to reimplement them in this scheme.]

The set of task operators includes: break-object, clean-object, close-container, cook-microwave, dirty-object, empty-object, fill-object, get-object, go-to, look-at, open-container, put-in, put-on, slice-object, turn-on, turn-off. The task operators' working memory structure and implementation rules are similar. One simple example is given: clean-object. The details of each operator are not provided except when there is something unusual about them.

The knowledge associated with a task action operator can be divided into a set of conceptual parts, each is described below. In general, a task action operator is implemented via two Soar rules.

- The first proposes the operator by creating an acceptable preference and the operator structure in working memory. The details are below, but these are MEA proposal rules. Some operators have multiple proposal rules because they can apply to achieve distinct desired outcomes.
- The second rule implements the actions of the operator during a look-ahead search.

That is it. Two rules.

A third rule can be included that proposes the operator whenever its preconditions are met, but these operators are not used in the current agent and were only written so that, at one point, a pure look-ahead search agent could be compared to an MEA agent. However, the agent is not *all in* on using MEA, so those rules, when they exist, have not been recently tested.

There are general, operator-independent rules that apply to an operator when it is selected by initiating its action on the output-link.

Task Action Operator Working Memory Structure

This section describes the operator's working memory structure that is created when the proposal operator fires. The complete rule is shown later.

The example operator is clean-object, which magically makes a dirty object clean.

```
   (<s> ^operator <operator> +)
   (<operator> ^name clean-object
```

```
                ^task-operator true
                ^action-handle clean
                ^super-desired <desired>
                ^argument1 <object>
                ^argument-count 1
                ^desireds <desireds2>)
   (<desireds2> ^desired-count 1
                ^desired.object <desired-object2>)
   (<desired-object2> ^reachable true
                      ^object-handle <clean-handle>)
```

- ^**name** [string] The name of the operator.
- ^**task-operator** [true] This distinguishes it from other operators that are not task operators, although its usage needs to be better defined.
- ^**action-handle** [string] This is the command that will be created on the output-link as part of the perform-action structure, but a support rule when the operator is selected, and its preconditions are satisfied.
- ^**super-desired** [identifier: desired] This allows access to information on why this operator is being selected to achieve. That is used by some generic search control knowledge in the spirit of differences from GPS.
- ^**argument1** [identifier: object] This is an argument for executing the operator. In this case, it is the object being cleaned. It will usually be a complete, grounded object, but all that is necessary is that the object it points to has the appropriate ^object-handle. There can be any number of arguments (argument2, argument3, ...), although currently there are no more than 3. In slice-object, the knife is not included as an argument, but must be held (which is tested in a precondition/desired for the operator).
- ^**argument-count** [integer] The number of arguments.
- ^**desireds** [identifier: desireds] The desireds structure (not desired) defines the pre-conditions of the operator as a set of ^desired structures.
  - ^**desired** [identifier: object/relation] These are the relations and object properties that must be true in the world for the operator to apply. There are separate rules in elaborations/achieved-desireds.soar that track whether they are achieved. When all desired structures are achieved, ^pre-conditions-satisfied true is added to the operator. A desired can flip back and forth from achieved to unachieved.
  - Not shown is that ^preferred-desired, ^avoid-desired, or indifferent-desired can be included on a ^desireds structure to provide search control information for which desired should be preferred or avoided. If there are ^preferred-desired, ^avoid-desired, or ^indifferent-desired for that desired, then a corresponding preference (>, <, =) is created

for selecting this operator. There *cannot be* ^next-desireds for ordering of sets of desireds.
    - To indicate whether a desired is achieved or not, a structure is added to the desireds object, such as (<desireds> ^achieved <desired>)
    - ^**desireds-count:** The total number of desired structures
    - ^**unachieved-desired-count:** How many desired structures are not yet achieved.
    - ^**achieved-desired-count:** How many desired structures are not yet achieved.
- ^**pre-conditions-satisfied** [true] True when all of the desired structures in desireds are satisfied. When true, task-independent rules create the appropriate structure on the output-link to perform the action. (external-operator-application.soar)

There are a few cases of specialized structures for task operators.

Task Operator Rules

Every operator has its own rules that include an MEA proposal rule and an internal implementation rule for planning or internal simulation. There can be a rule for non-MEA proposal, but it is not required. Because of the uniform working structure of the operators (see above), generic rules can create the correct structures on the output-link (external-operator-application.soar) and a few other housekeeping activities related to look-ahead search. Here, I use the example of clean-object to describe the first two types of rules and their components. Many operators require additional rules, and some of those are included below under their name, ordered alphabetically.

Clean-object

```
sp {thor-soarproposeclean-object*mea
   (state <s> ^problem-space.name thor-world
              ^flags.mea-operator-selection true
              ^world <world>
              ^desireds <desireds>)
   (<world> ^anchored-world true
            ^objects.object <object>)
   (<object> ^<< origin-object-handle object-handle >> <o-clean-handle>
             ^object-handle <clean-handle>)
   (<desireds> ^desired <desired>
               ^unachieved <desired>)
   (<desired> ^object <desired-object>)
   (<desired-object> ^dirty false
                     ^object-handle <o-clean-handle>)
-->
```

```
   (<s> ^operator <operator> +)
   (<operator> ^name clean-object
               ^task-operator true
               ^action-handle clean
               ^super-desired <desired>
               ^argument1 <object>
               ^argument-count 1
               ^desireds <desireds2>)
   (<desireds2> ^desired-count 1
                ^desired.object <desired-object2>)
   (<desired-object2> ^reachable true
                      ^object-handle <clean-handle>)
}
```

- MEA selection:
  - **^problem-space.name** [thor-world]
  - **^flags.mea-operator-selection** [true] This is tested so that this rule only fires when using MEA.
  - ^**world** [identifier: world] The world identifier provides a path to an object (in this case) or a relation (on/in) so that the desired structure can be grounded in the world.
    - In this example, it provides a link to ^objects.object <object>, which is an object in the world that shares the object-handle with the desired object. The disjunction: ^<< origin-object-handle object-handle >> is required because some objects are derived from others (slices of a potato), and the original desired might include the object-handle of the potato, and not the slice that is supposed to be cleaned, because it does not yet exist. This is confusing...
    - It also tests ^anchored-world true to distinguish it as either the real world or a hypothetical real world being explored in look-ahead. If it is false or not included, the world and its substructures are derived from a world-context structure that provides constraints on the grounding of objects, but is not a complete world.
  - ^**desireds** [identifier: desired] This describes the properties or relations that this operator is proposed to achieve.
    - There is usually only one ^**desired**. In this case, the desired is that there is an object with object-handle <break-object-handle> that has property ^dirty false (meaning it is clean). This is not a precondition, but a property to be achieved by the operator. As stated earlier, the desired *should not* be an object or relation (on/in) structure in the ^world. It describes how an object or relation should be changed, and links to structures in the ^world via ^object-handle.

        - ^unachieved <desired>is included so that the operator is proposed only if the desired is not achieved.
- Internal Application: This is the rule(s) that apply the operator to internal models during look-ahead. It is not used when the operator is applied in the real world. For that, generic rules use the action-handle and argument1/2 structures to create perform-action structures on the output-link.
    - **^internal-actions** [true] This signifies that the operator is being applied to an internal model in look-ahead.
    - **^operator** [identifier: operator] The operator is matched along with any arguments
        - **^pre-conditions-satisfied** [true] This is tested so that the operator is applied only when the pre-conditions are satisfied.
    - The actions:
        - I always provide a log of an operator applying. This is important because operators in MEA/operator substate stacks apply only when the preconditions are finally satisfied, which can be many decisions after they have been selected.
        - The action is to make the object clean.

```
sp {applyclean-objectinternal
   (state <s> ^operator <operator>
              ^internal-actions true)
   (<operator> ^name clean-object
               ^argument1 <object>
               ^pre-conditions-satisfied true)
   (<object> ^object-handle <object-handle>
             ^dirty true)
-->
   (log 1 (crlf) |Apply clean: | <object-handle> | internal.|)
   (<object> ^dirty true -
                     false)
}
```

Turn-on:

Turn-on is special because it creates future-events on the world. There are two proposals. One is the standard one used in all cases except when a specific time is set for how long something should be turned on. So far, that is unique to the microwave.

There are separate operator applications for each type of “machine” that can be turned on: microwave, coffee-machine, stove-knob, toaster (the dishwasher is currently instantaneous).

Break:

When an egg is broken, the original object should be destroyed, but it isn't, so there is an operator finish-broken-egg that does that. It has an argument2 that is the object handle of the new object. It has desireds that the object being broken is reachable (close by) and accessible (not in a closed container). It specifies that being accessible should be achieved first.

There is a second proposal for when an egg is desired to be cooked. The egg must be broken before it is cooked. Seems like this should be rewritten so its is a desired for cooking an egg.

Because if a bug in breaking the egg, there is an operator application rule for external breaks. Too complex to describe here. Similarly, for internal breaks, it is complicated because a new object is created and all of the features of the original object must be copied to the new object.

Need to update the below operator (12/1/2025)

Close-container

Need to be careful about closing drawers - ...

Empty-object

Nothing special

Fill-object

- Has separate non-mea proposals for each type of liquid (water, wine, coffee)
- Has one mea proposal for when a specific liquid is specified, and one for when no specific liquid is specified (^contains-liquid true), then it tries to fill with water. It could be more sophisticated to fill with the most easily accessible liquid, whatever that means.

Get-object

- It has two sets of rules (six total), one for if the object is in a container and one for if it is on a location. It could be split into two operators, but this works...

Go-to

- It does not use action-handle approach, but has rules for external applications because it needs to handle three cases. Object is on a location, object is in a container, object is neither. Although looking at the current rules, I'm not sure this is necessary as they all seem to approach the same object. Probably can rewrite.
- Separate internal implementation operators to manage changes in reachable properties. This is not always sufficient because not all objects that will be reachable after movement are attended to in the world.

Open-container

- Has separate proposals for containers that are activatable (must not be activated) and those that are not.
- Rejects trying to open a drawer if also trying to go-to or get-object and have an object in the hand because will get stuck with the drawer open. [Need to explain this better]

Slice-object

- Internal apply is hilarious because it creates multiple (3) sliced objects and destroys the original object. Not all objects end up sliced into three (apple = 3, potato = 6), but I'm lazy.

Turn-on

Toggle-object

- Separate proposals and internal applications for toggling on vs. off.
- MEA proposals work for << activated toggled >>
- Assumes must have empty gripper to toggle/activate (which might not be true in AI2-Thor.

Non-Task Operators

There are various operators that perform supporting internal reasoning actions, not involving issues commands on the output-link. Listed in alphabetical order

Need to update the below operator (12/1/2025)

Attend-object

If there is a desired that does not have a define object-handle - it has (object> ^object-handle-undefined true), then looks in the scene for an object with the attribute and value of that object and attends to the object, which will cause it to be added to the world. Not currently used, so not tested. Will have issues if there are more than one property specified for the desired object.

Create-object-state-desired

If flags.clean-up-up-success true, then this creates a desireds structure on the top state when the original desireds is achieved. The created desired is to close doors and deactivate objects (turn of the faucet), except for the lights. The desireds properties are specified in constants.desired-object-states. Should not be used if an ongoing interaction with a human, but fun for testing.

Detect-illegal-desired

This operator is proposed whenever an unachievable desired is creation, such as putting something in a surface, on a receptacle, cooking something that is not cookable, etc. The operator adds <desireds> ^illegal-desired <desired> and interrupts. Probably want to remove the interrupt once there is code to detail with the ^illegal-desired.

Halt

If flags.desired-source elaboration, and desireds is success, then interrupt. Useful for JEL when testing agent.

Init-internal-simulation

Currently not active, but the idea is to initialize the world with an fake internal state and then perform internal simulation problem solving.

Init-thor-soar

First operator selected and applied, creates top-state agumentations include world, scene, attends, constants. Flags are created in elaborations/flag-initialization.soar

Next-desireds

When the current desireds on a state have success, and there is a ^next-desireds, it makes that the current desireds.

Remove-object

I flags.forget-unattended true, remove an object from the world if it is no longer attended. Theoretically, keeps world small. Not tested.

Svs-extract

Not sure. SVS extraction is handled by elaborations/svs.soar.

Elaboration Files

Means-Ends Analysis Problem Solving

Selection Space/Evaluate Operator Iterative Deepening Look-Ahead Search

Chunking